

ATIVIDADE_007_DO_VINICIUS_GIRATO_MORESQUI_RA_26014779-2

Módulo 1: Expressões, Precedência e Efeitos Colaterais (Slide 1 a 7)

Objetivo da aula - Tem como objetivo entender como as linguagem avaliam expressões e controlam o fluxo de execução, onde de marcamos o que código faça corretamente o que ele apresenta na execução, sem surpresas de procedência ou desvios. Para um eng. de software isso vai representar ler código alheio sem ser enganado pela indentação, evitar conversões implícitas perigosas e escolher a estrutura de controle mais legível para cada caso.

Quiz de abertura:

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 - Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 - Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

1 - `print (-2 ** 2, 2 ** 3 ** 2)` - Palpite: o primeiro resultado vai dar -4 e segundo vai ser 500 e pouco. Isso ocorre pois a vírgula vai dividir o cálculo em dois.

2 - `System.out.println ( 1 + 2 + "3" + 4 + 5);` - Palpite: vai ocorrer um erro no compilador, onde `println` ele apenas mostra resultado dentro das aspas, aqui ele está tentando valores foras desse

Precedência e Associatividade de Operadores:

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

`a + b * c` com `a=3, b=4, c=5`

da esquerda para a direita

(3 + 4) * 5 = 35

da direita para a esquerda

Por exemplo, a multiplicação (*) tem precedência sobre a adição (+), e a associatividade define a direção do cálculo (a maioria vai da esquerda para a direita, mas o expoente ** em Python e JS vai da direita para a esquerda).

A Ordem dos Operandos e Efeitos Colaterais:

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	–
<code>1 + 2 + "3"</code>	<code>TypeError</code>	"33"	"33"
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` / `%` + `-`

Direita: `**` (Python, JS, Ruby, Fortran), `++`, `--`, menos unário

Visual Basic: `^` à esquerda!

Excel

`=-2^2` dá 4: lá o menos unário vence o `^`.

a ordem em que os operandos de uma expressão são lidos **não é garantida** pelo padrão. Isso significa que se você tiver uma expressão com uma função que altera uma variável global (um efeito colateral, como `a + fun1()`), o resultado pode mudar dependendo de qual parte a linguagem decide calcular primeiro, gerando bugs difíceis de achar.

Sobrecarga de Operadores:

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de `a` em relação à chamada

símbolo pode ter vários significados dependendo do contexto ou da linguagem (por exemplo, o `+` que soma números ou concatena textos). Além disso, mostra que algumas linguagens permitem que o programador crie seus próprios significados para operadores em classes personalizadas (como somar dois vetores usando o operador `+` em Python), enquanto outras linguagens proíbem isso.

Módulo 2: Tipos, Coerção e Expressões Booleanas

Conversões de Tipos:

O que valida: Valida como o compilador lida com a mistura de tipos numéricos (como `int` e `float`).

O que comprova: Comprova que o **alargamento** (converter um número menor para um maior, como `int` para `float`) é seguro e geralmente feito de forma implícita pelas linguagens.

Conversões de tipos: alargamento, estreitamento e coerção

Java • coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Coerção implícita:

O que valida: Valida os perigos das linguagens que tentam "adivinhar" o que você quer fazer quando você mistura tipos diferentes em uma operação.

O que comprova: Comprova que conversões automáticas (coerção) geram armadilhas bizarras e bugs de segurança perigosos — como o famoso erro do PHP em que strings de hash começando com `0e` eram comparadas com `==` e consideradas iguais (enganando sistemas de login).

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + "a" + "a" // "baNaNa"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO")   // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que não fazem coerção.

Expressões relacionais e booleanas:

O que valida: Valida a forma como diferentes linguagens definem o que é verdadeiro ou falso em estruturas condicionais (como o comando **if**).

O que comprova: Comprova que linguagens mais antigas ou flexíveis (como C, Python e JavaScript) aceitam qualquer valor em um **if** (considerando números zero, strings vazias ou listas vazias como falso).

Expressões relacionais e booleanas

C • booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python • comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0  0.0  ""  []  {}  None
# contam como VERDADEIRO:
# "0"  [0]  " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. - §7.5 - p. 313-315 - exemplos/05_booleanas

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade    // Retorna: undefined, sem dar erro const
qtd = 0; qtd || 10    // Retorna: 10 (o 0 se perdeu!)
qtd ?? 10            // Retorna: 0 (só troca null ou undefined)
```

```
const u = { nome: "Ana", endereco: null };
```

Cria um objeto constante chamado **u** com duas propriedades: **nome** (que guarda a string "Ana") e **endereco** (que está definido explicitamente como **null**).

```
u.endereco?.cidade
```

Tenta aceder à propriedade **cidade** dentro do objeto **endereco**. Como **endereco** é **null**, tentar aceder a uma propriedade de algo nulo normalmente faria o programa falhar (dar erro de execução). No entanto, graças ao **encadeamento opcional (?)**, a expressão faz um curto-circuito de segurança: ela para imediatamente e devolve **undefined** em vez de quebrar a aplicação.

```
const qtd = 0;
```

Define uma variável numérica chamada **qtd** com o valor **0**.

```
qtd || 10
```

Usa o operador lógico "OU" (**||**). Em JavaScript, o número **0** é considerado um valor "falso" (*false*) nas avaliações booleanas. Como o lado esquerdo é **0**, o operador assume que ele é inválido/falso e recorre ao lado direito, devolvendo **10** (ou seja, o valor **0** original perdeu-se).

qtd ?? 10

Usa o operador de coalescência nula (`??`). Ao contrário do operador `||`, este operador é mais rigoroso: ele só substitui o valor da esquerda se este for estritamente `null` ou `undefined`. Como `0` é um número válido (e não nulo), a expressão respeita o valor e devolve `0`.

Módulo 3: Sentenças de Atribuição e Estruturas de Seleção (Slides 12 a 16).

Atribuição como Expressão:

O que valida: Analisa como diferentes linguagens tratam o comando de atribuição (`=`).

O que o conteúdo explica: Em linguagens como C, C++ e JavaScript, a atribuição é considerada uma expressão que retorna um valor. Isso permite armadilhas clássicas de digitação, como escrever `if (x = y)` (onde você atribui `y` a `x` e testa se o resultado é verdadeiro) em vez de comparar `if (x == y)`.

Como as linguagens modernas ajudam: Linguagens como Python, Go e Rust proíbem que a atribuição seja usada dessa forma em condicionais, evitando que bugs silenciosos aconteçam. O Python introduziu o operador morsa (`:=`) para casos controlados onde a atribuição dentro de uma expressão é realmente necessária.

Atribuição Mista e Coerção na Atribuição:

O que valida: Como a linguagem lida quando misturamos tipos diferentes em uma operação de atribuição.

O que o conteúdo explica:

- O **Java** permite apenas o alargamento implícito (ex: colocar um inteiro dentro de um `double`) ou exige cast explícito para estreitamentos.
- O **C** faz coerção em qualquer direção, o que pode gerar perda de dados em silêncio (como colocar `300` em um `char`).
- **A armadilha clássica:** Fazer cálculos de porcentagem dividindo números inteiros antes da conversão (ex: `double taxa = acertos / total * 100;`).

O Problema do *Else Pendente*:

O que valida: A ambiguidade estrutural em estruturas condicionais aninhadas de dois caminhos (*if/else*).

O que o conteúdo explica: Em linguagens como C, C++ e Java, a indentação visual do código é completamente ignorada pelo compilador. Se você tem um *if* dentro de outro *if* sem chaves, o *else* sempre pertencerá ao *if* mais próximo que ainda não tem um *else*, o que frequentemente gera comportamentos indesejados.

Como as linguagens resolvem: Linguagens mais seguras (como Go, Rust e Perl) tornam o uso de **chaves obrigatórias** em blocos condicionais, eliminando essa ambiguidade. Além disso, linguagens modernas permitem que o *if* funcione como uma expressão que devolve um valor diretamente (como em Rust e Python).

Módulo 4: Laços, Desvios e Confiabilidade (Slides 17 a 20).

Laços Controlados por Contador e Lógicos:

O que valida: Compara a forma como diferentes linguagens gerenciam repetições e contagens.

O que o conteúdo explica:

- O *for* tradicional do C funciona mais como um laço lógico disfarçado, permitindo alterar a variável de contagem no meio do bloco ou omitir expressões, o que abre margem para erros.
- Em contraste, o *for* do Python percorre um iterável (como o *range*), onde alterar a variável de controle no corpo do laço não afeta o fluxo das iterações.

O perigo do JavaScript (*for...in* vs *for...of*): O material destaca que usar *for...in* em arrays no JavaScript percorre as chaves (índices) e pode causar resultados inesperados (como concatenar textos em vez de somar números), enquanto o *for...of* percorre corretamente os valores.

O Perigo de Laços sem Progresso:

O que valida: A importância crítica de garantir que todo laço lógico possua uma condição de progresso em direção ao término.

O que o conteúdo explica: Analisa o bug histórico de 2008 nos tocadores Zune da Microsoft. Devido a uma falha num laço *while* que calculava os dias

em anos bissextos, quando o número de dias restantes era exatamente 366, a variável não sofria alteração, criando um **laço infinito** que travou todos os aparelhos do mundo no mesmo dia.

Desvio Incondicional (**goto**):

O que valida: Por que o comando **goto** foi duramente criticado por Edsger Dijkstra em 1968 ("*Go To Statement Considered Harmful*") e os perigos de desvios desestruturados de fluxo.

O que o conteúdo explica:

- O **goto** permite saltar para qualquer parte do código, destruindo a legibilidade e a previsibilidade.
- Analisa o famoso bug da Apple em 2014 ("**goto fail**"), onde uma linha **goto fail**; duplicada por engano fazia o código saltar a verificação de certificados de segurança, aceitando certificados falsos no iOS e macOS.
- O texto aponta que estruturas restritas como **break**, **continue** e **return** mantêm a legibilidade sem os riscos do **goto**.

Comandos Protegidos de Dijkstra:

O que valida: O conceito de escolhas não determinísticas e execução baseada em **guardas** (condições lógicas).

O que o conteúdo explica: Nos comandos protegidos originais de Dijkstra, várias condições poderiam ser verdadeiras ao mesmo tempo, e a escolha de qual caminho seguir não era determinística. Esse conceito evoluiu e inspirou recursos de concorrência modernos, como o comando **select** da linguagem Go, que sorteia e executa canais de comunicação que estejam prontos no momento.

Atividade

Exercício: encontre o bug (2/3)

Para cada trecho: (a) qual é a saída? (b) qual regra causa o problema? (c) como corrigir?

3 - Java

```
int dia = 2; String nome = "";
switch (dia) {
    case 1: nome = "domingo";
    case 2: nome = "segunda";
    case 3: nome = "terça";
    default: nome = "inválido";
}
System.out.println(nome);
```

4 - C

```
int saldo = 100, saque = 50;
if (saque > saldo);
    printf("saldo insuficiente\n");
printf("fim\n");
```

Arquivos: exemplos/10_exercicio (q1.js ... Q6.java) - execute em onecompiler.com

3 - Java

(a) Qual é a saída?

A saída impressa na consola será inválido.

(b) Qual regra causa o problema?

Ocorre o efeito de *fallthrough* (queda). No switch tradicional herdado do C, quando um caso é correspondido (case 2), o código executa essa linha, mas continua a descer executando os casos seguintes em sequência (case 3 e default), sobrescrevendo a variável nome até chegar ao fim.

(c) Como corrigir?

Adicionar o comando break; no final de cada case para interromper o fluxo assim que o bloco correspondente for executado, ou utilizar a sintaxe moderna do switch em Java (->).

4 - C

(a) Qual é a saída?

A saída será:

saldo insuficiente
fim

(b) Qual regra causa o problema?

O ponto e vírgula ; colocado imediatamente após o fecho dos parênteses do if (saque > saldo); cria um comando vazio. Isto significa que o if termina ali mesmo. A linha do printf("saldo insuficiente\n"); fica totalmente solta e independente da condição, executando sempre. A endentação visual engana o programador, dando a falsa sensação de que o printf está protegido pelo if.

(c) Como corrigir?

Remover o ponto e vírgula logo após o parêntese do if e utilizar chaves {} para delimitar explicitamente o bloco de código condicional.